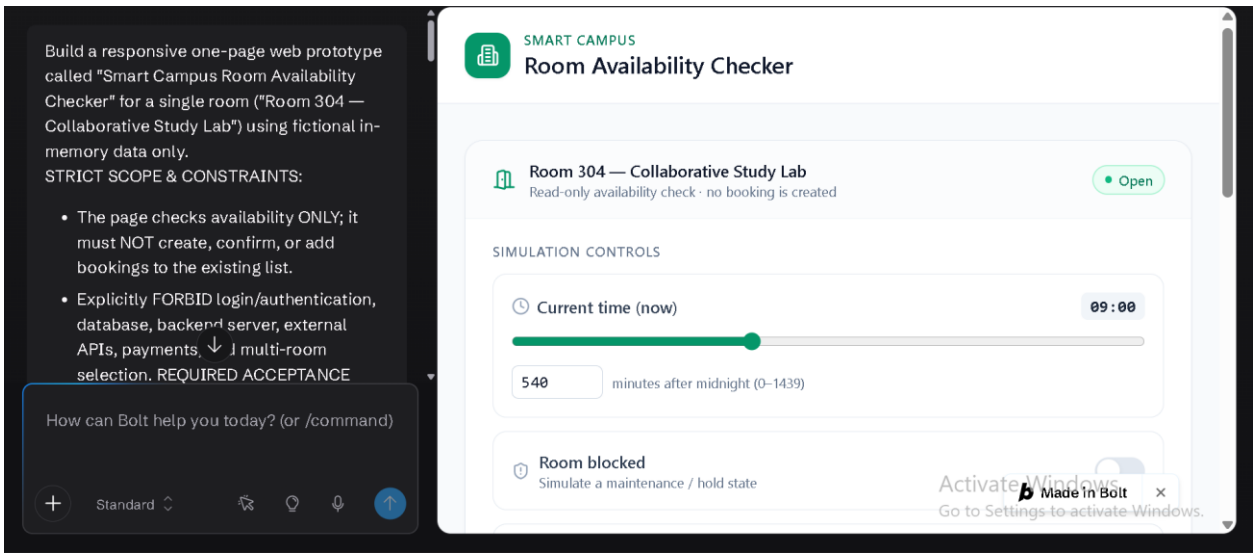
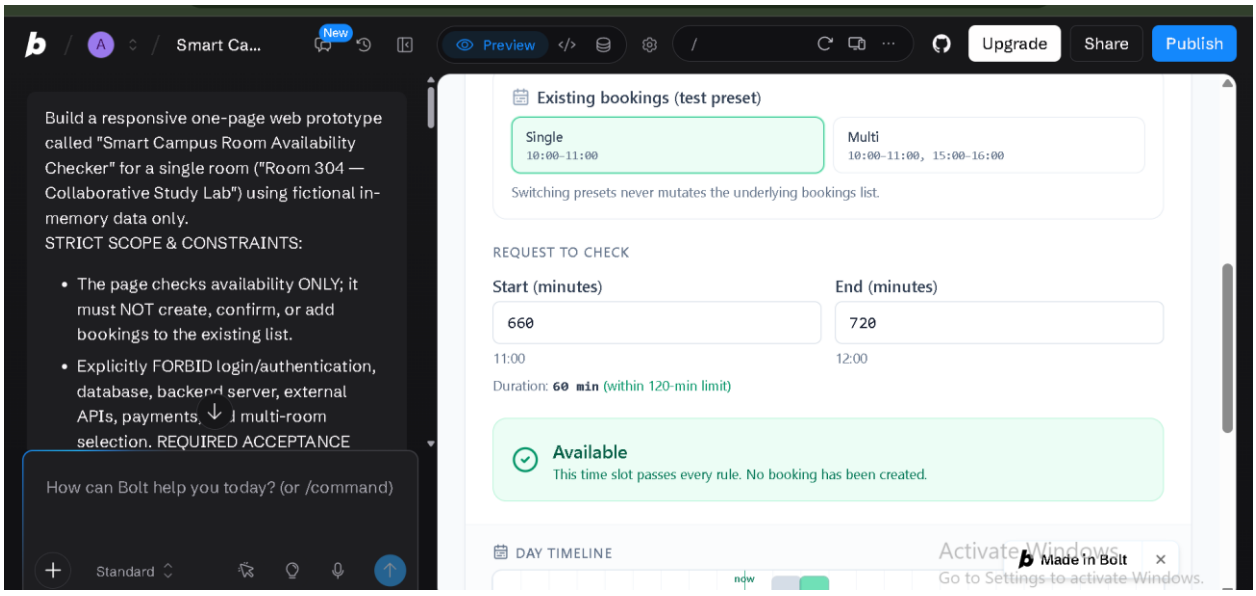
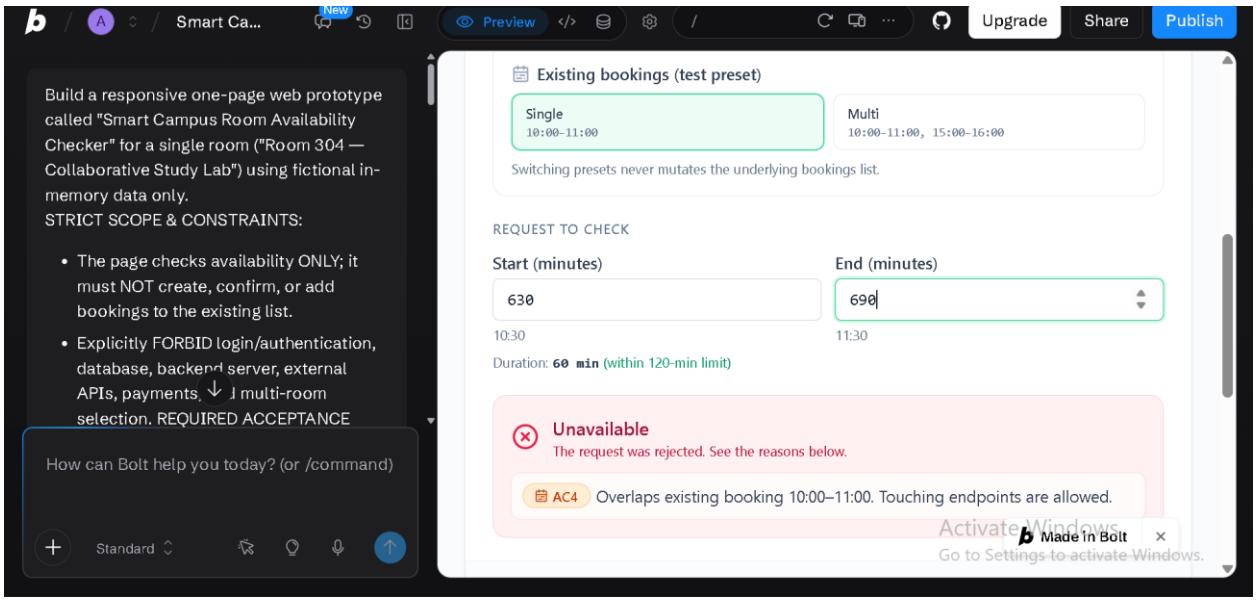
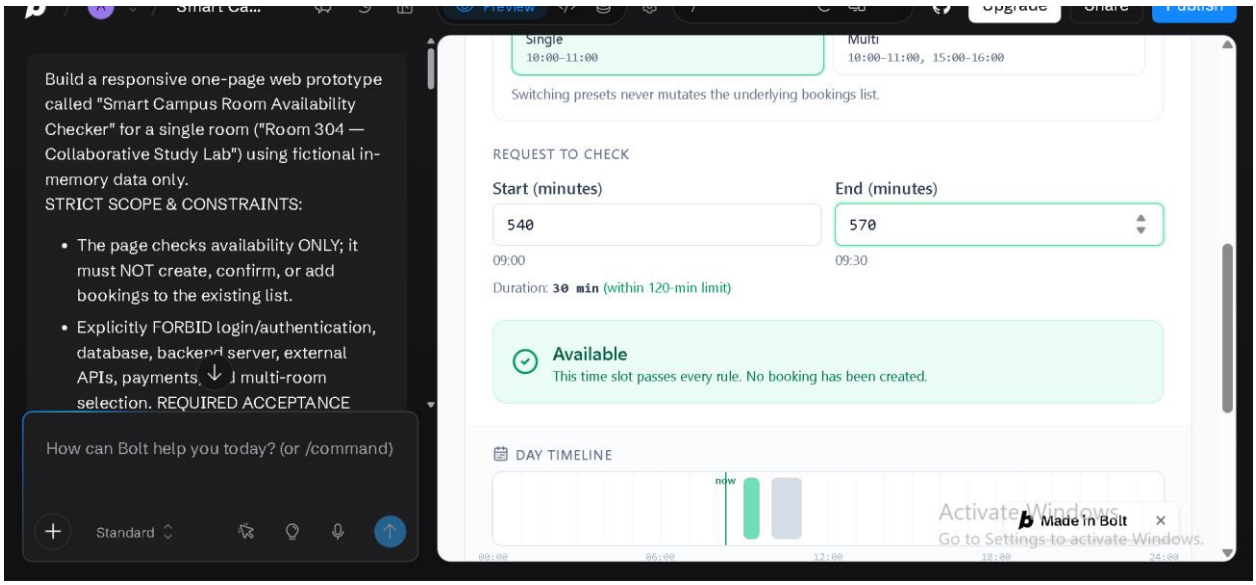
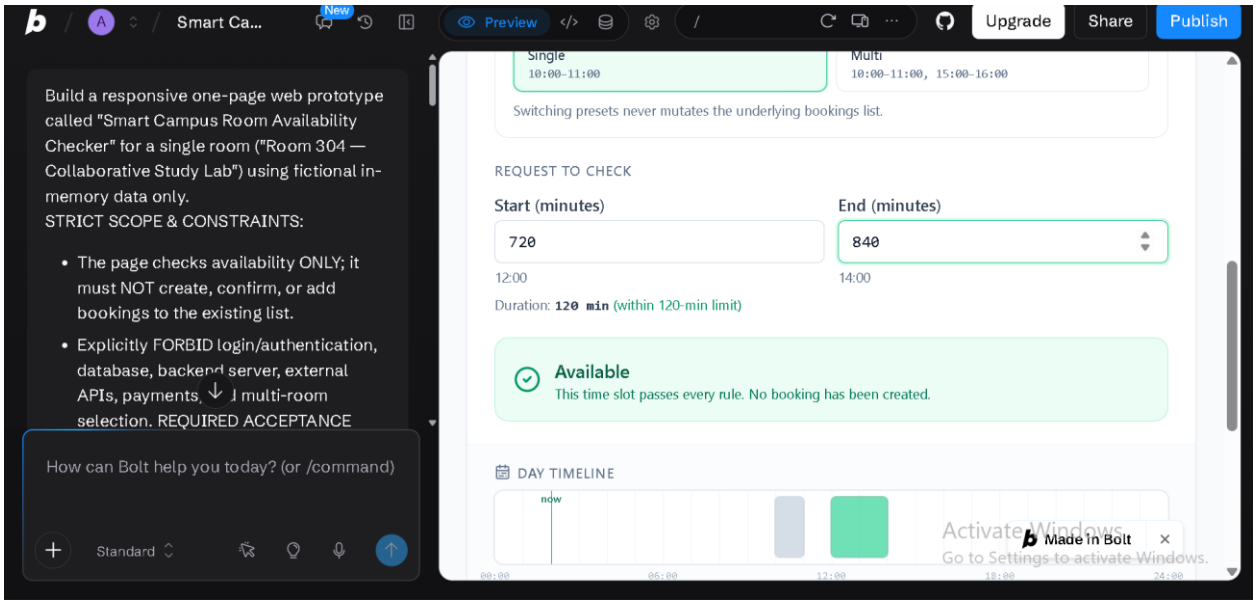
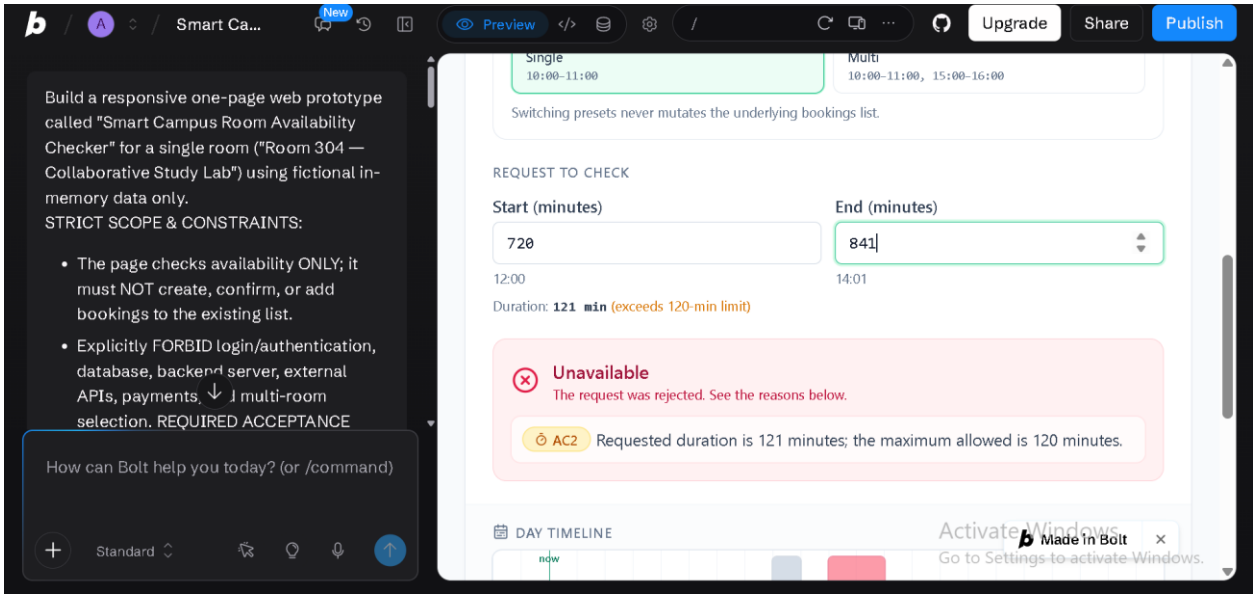

Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
SMART CAMPUS
Room Availability Checker
Room 304 — Collaborative Study Lab
Read-only availability check · no booking is created
Open
SIMULATION CONTROLS
Current time (now)
09:00
540
minutes after midnight (0–1439)
Room blocked
Simulate a maintenance / hold state
Made in Bolt
Activate Windows
Go to Settings to activate Windows.


Smart Ca...
New
Preview
Upgrade
Share
Publish
Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
Existing bookings (test preset)
Single
10:00–11:00
Multi
10:00–11:00, 15:00–16:00
Switching presets never mutates the underlying bookings list.
REQUEST TO CHECK
Start (minutes)
End (minutes)
660
720
11:00
12:00
Duration: 60 min (within 120-min limit)
Available
This time slot passes every rule. No booking has been created.
DAY TIMELINE
Made in Bolt
Activate Windows
Go to Settings to activate Windows.

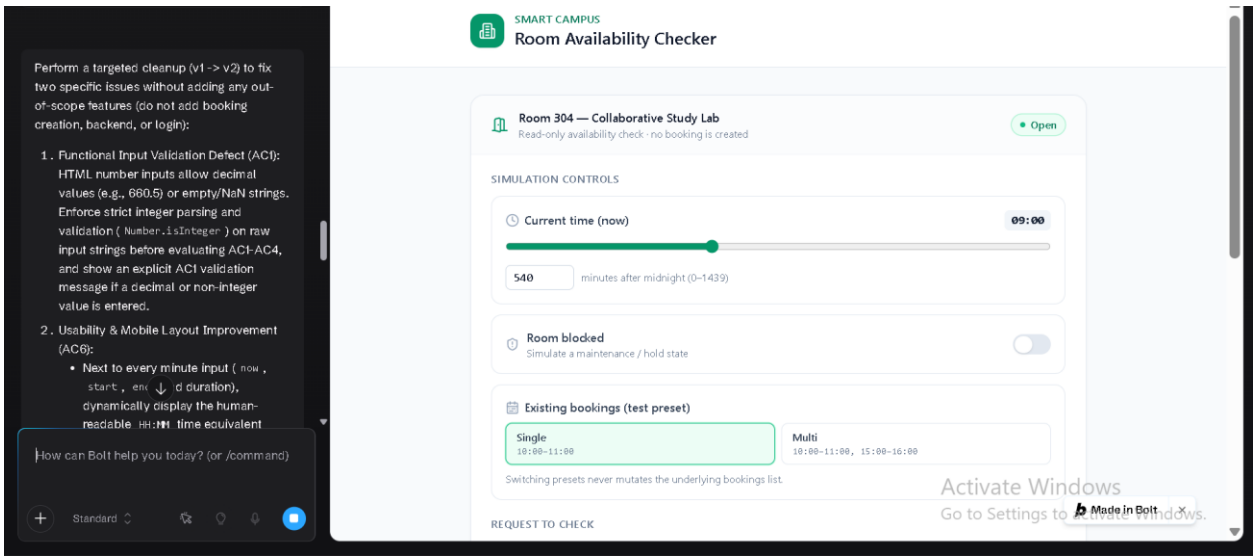

Smart Ca...
New
Preview
Upgrade
Share
Publish
Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
Existing bookings (test preset)
Single
10:00–11:00
Multi
10:00–11:00, 15:00–16:00
Switching presets never mutates the underlying bookings list.
REQUEST TO CHECK
Start (minutes)
End (minutes)
630
690
10:30
11:30
Duration: 60 min (within 120-min limit)
Unavailable
The request was rejected. See the reasons below.
AC4
Overlaps existing booking 10:00–11:00. Touching endpoints are allowed.
Made in Bolt
Activate Windows
Go to Settings to activate Windows.


Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
Switching presets never mutates the underlying bookings list.
REQUEST TO CHECK
Start (minutes)
End (minutes)
540
570
09:00
09:30
Duration: 30 min (within 120-min limit)
Available
This time slot passes every rule. No booking has been created.
DAY TIMELINE
now
Made in Bolt


Smart Ca...
Preview
Upgrade
Share
Publish
Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
Switching presets never mutates the underlying bookings list.
REQUEST TO CHECK
Start (minutes)
End (minutes)
720
840
12:00
14:00
Duration: 120 min (within 120-min limit)
Available
This time slot passes every rule. No booking has been created.
DAY TIMELINE
now
Made in Bolt


Smart Ca...
Preview
Upgrade
Share
Publish
Build a responsive one-page web prototype called "Smart Campus Room Availability Checker" for a single room ("Room 304 — Collaborative Study Lab") using fictional in-memory data only.
STRICT SCOPE & CONSTRAINTS:
• The page checks availability ONLY; it must NOT create, confirm, or add bookings to the existing list.
• Explicitly FORBID login/authentication, database, backend server, external APIs, payments, and multi-room selection. REQUIRED ACCEPTANCE
How can Bolt help you today? (or /command)
Standard
Switching presets never mutates the underlying bookings list.
REQUEST TO CHECK
Start (minutes)
End (minutes)
720
841
12:00
14:01
Duration: 121 min (exceeds 120-min limit)
Unavailable
The request was rejected. See the reasons below.
AC2
Requested duration is 121 minutes; the maximum allowed is 120 minutes.
DAY TIMELINE
now
Made in Bolt

V2




Activate Windows
Go to Settings to activate Windows.

mart campus room booking validation plan

Share

I am working on Software Engineering Assignment #03: Smart Campus Room Availability Checker.
Before generating any code, I need a concise engineering plan (Stage A: Plan) for implementing `can_book(start, end, now, blocked, existing)` in `booking.py` and unit tests in `test_booking.py` using only the Python 3 standard library.

Fixed Acceptance Criteria:
- AC1: Candidate times are integer minutes after midnight: 0 <= start < end <= 1440; `start` must be strictly greater than `now` (assume `now` is valid in 0..1439).
- AC2: Duration (`end - start`) must not exceed 120 minutes. Exactly 120 minutes is permitted.
- AC3: Reject every request when `blocked` is True.
- AC4: Reject overlap with any interval `(s, e)` in `existing`. Intervals touching at an endpoint (e.g., 600-660 and 660-720) do NOT overlap.
- AC5: Return True only when valid, False otherwise, and do NOT mutate `existing`.

Please provide ONLY:
1. A step-by-step validation plan for `can_book`.
2. Explicit assumptions (including Python type-checking nuances).
3. A list of boundary and edge test cases to include in `test_booking.py`.
Do not generate the final implementation code yet.

Show less

4 hours ago

## Stage A: Engineering Plan for `can_book(start, end, now, blocked, existing)`

### 1. Step-by-step validation plan

Checks run in a fixed order and short-circuit to `False` on the first failure. Cheap checks come first, and `existing` is only scanned once everything else has passed.

1. **Type validation (AC1, AC5).** Reject if `start`, `end` or `now` is not a true `int`. This includes rejecting `bool` (see assumptions). If `blocked` is not a `bool`, treat it as invalid.
2. **Blocked check (AC3).** If `blocked is True`, return `False`. This comes early because it overrides every other condition.
3. **Range and order check (AC1).** Require `0 <= start < end <= 1440`. This also guarantees `end - start >= 1`.

Activate Windows
Go to Settings to activate Windows.

Your free messages return at 1:20 AM.

See Pro plan

4 hours ago

## Stage A: Engineering Plan for `can_book(start, end, now, blocked, existing)`

### 1. Step-by-step validation plan

Checks run in a fixed order and short-circuit to `False` on the first failure. Cheap checks come first, and `existing` is only scanned once everything else has passed.

1. **Type validation (AC1, AC5).** Reject if `start`, `end` or `now` is not a true `int`. This includes rejecting `bool` (see assumptions). If `blocked` is not a `bool`, treat it as invalid.
2. **Blocked check (AC3).** If `blocked is True`, return `False`. This comes early because it overrides every other condition.
3. **Range and order check (AC1).** Require `0 <= start < end <= 1440`. This also guarantees `end - start >= 1`.
4. **Future-start check (AC1).** Require `start > now` (strict). `start == now` is rejected.
5. **Duration check (AC2).** Require `end - start <= 120`. Exactly 120 passes.
6. **Overlap check (AC4).** For each `(s, e)` in `existing`, the candidate conflicts if `start < e and s < end`. Strict inequalities make touching endpoints (`e == start` or `s == end`) non-overlapping. Return `False` on the first conflict.
7. **Success (AC5).** Return `True`.

The function is read-only with respect to `existing`. It iterates over it but never calls `sort`, `append`, `remove` or any other mutating method, and it does not reassign or alias it into something that gets modified.

### 2. Explicit assumptions

**Inputs**

- `now` is valid (0..1439), as stated in the assignment, so it is not re-validated for range.
- `existing` is an iterable of `(s, e)` pairs. Each pair is a 2-element tuple or list of ints with `s < e`, and the intervals are mutually valid.
- `existing` may be empty (and `None` is not a supported value).
- Existing intervals are not assumed to be sorted or non-overlapping with each other, so a full scan is used instead of binary se

Activate Windows
Go to Settings to activate W

Your free messages return at 1:20 AM. See Pro plan

Claude is AI and can make mistakes. Please double-check responses.

---

**AC2: duration**

- Duration 119 returns `True`.
- Duration 120 returns `True`.
- Duration 121 returns `False`.
- Duration 1 returns `True`.

**AC3: blocked**

- `blocked=True` with otherwise perfectly valid request and empty `existing` returns `False`.
- `blocked=True` with an otherwise invalid request returns `False`.
- `blocked=False` with valid request returns `True`.

**AC4: overlap**

- Existing `(600, 660)`, candidate `(600, 660)` (identical) returns `False`.
- Candidate fully inside existing, e.g. `(610, 650)` returns `False`.
- Candidate fully containing existing, e.g. `(590, 670)` returns `False`.
- Partial overlap at start, e.g. `(570, 630)` returns `False`.
- Partial overlap at end, e.g. `(630, 690)` returns `False`.
- Touching: existing `(600, 660)`, candidate `(660, 720)` returns `True`.
- Touching: existing `(660, 720)`, candidate `(600, 660)` returns `True`.
- One-minute overlap at either edge, e.g. candidate `(599, 601)` or `(659, 661)` returns `False`.
- Candidate sandwiched exactly between two existing intervals `(540, 600)` and `(660, 720)` with candidate `(600, 660)` returns `True`.
- Conflict only with the last item of a multi-item `existing` list ret
  (ensures the full scan).
- Conflict with an unsorted `existing` list returns `False`.
- Multiple non-overlapping existing intervals elsewhere returns
- Empty `existing` returns `True`.

**AC5: return type and non-mutation**

- Return value is a `bool` (use `assertIs(result, True/False)`

Your free messages return at 1:20 AM.

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share

Activate Windows
Go to Settings to activate Windows

- Empty `existing` returns `True`.

**AC5: return type and non-mutation**

- Return value is a `bool` (use `assertIs(result, True/False)`).
- Take a copy of `existing` before the call (`deepcopy` or `list(...)`), call `can_book` for both a `True` outcome and a `False` outcome, and assert the list is unchanged (same contents and order, including an unsorted list).
- If a tuple of tuples is used for `existing`, confirm it works without error.

**Type and invalid-input edge cases**

- `start` as float (`600.0`), string, or `None` returns `False`.
- `bool` passed as `start`, `end` or `now` (e.g. `True`) returns `False`.
- Non-bool `blocked` (e.g. `1`, `"yes"`) returns `False` (documented behavior).

**Combined and interaction cases**

- Valid duration and no overlap, but `start <= now` returns `False`.
- No overlap, but duration 121 returns `False`.
- Blocked plus overlap plus bad duration returns `False` (all failures at once).

---

Once you confirm or adjust these assumptions (notably the `bool` and non-bool `blocked` handling, and returning `False` rather than raising on bad types), I can move on to Stage B and generate the implementation and tests.

4 hours ago

I approve this plan. Now proceed to Stage B (Generate):
1. Provide the complete Python 3 implementation of `booking.py` containing `can_book(start, end, now, blocked, existing)` using only the standard library.
2. Provide the complete `test_booking.py` using `unittest` covering all required test cases:
- Base data: `now = 540`, `blocked = False`, `existing = [(600, 660)]`
- Touching: 660-720 -> True
- Overlap: 630-690 -> False
- Blocked: 660-720 with `blocked = True` -> False

Show more

Activate Windows
Go to Settings to activate Windows

I approve this plan. Now proceed to Stage B (Generate):
1. Provide the complete Python 3 implementation of `booking.py` containing `can_book(start, end, now, blocked, existing)` using only the standard library.
2. Provide the complete `test_booking.py` using `unittest` covering all required test cases:
- Base data: `now = 540`, `blocked = False`, `existing = [(600, 660)]`
- Touching: 660-720 -> True
- Overlap: 630-690 -> False
- Blocked: 660-720 with `blocked = True` -> False
- Exactly 120 min: 720-840 -> True
- 121 min: 720-841 -> False
- Starts now: 540-570 -> False
- Reversed times (e.g., 700-680), equal start/end (700-700), day bounds (< 0 and > 1440, exact 1440 end), multiple existing bookings, and verification that `existing` is not mutated after function execution.
3. Explain each conditional branch in `can_book` and map it directly to AC1-AC5.

Show less

4 hours ago

Created 2 files, ran a command, shared files >

I ran `python3 -m unittest -v` and all 40 tests pass. Both files, `booking.py` and `test_booking.py`, use only the standard library.

**Branch-to-AC mapping for `can_book`**

| # | Branch | Returns `False` when | Maps to |
|---|---|---|---|
| 1 | `_is_int(start/end/now)` | Any of the three is not a true `int`, including `bool`, float, str or `None`. | AC1 (times must be integers), AC5 (return |
| 2 | `type(blocked) is not bool` | `blocked` is not a real `bool`, e.g. `1` or `"yes"`. | |
| 3 | `if blocked` | `blocked` is `True`. | |
| 4 | `0 <= start < end <= 1440` fails | `start` `<`, `end > 1440`, or `start >= end`, which | |

Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Activate Windows
Markup and share.

| | | | |
|---|---|---|---|
| 6 | `end - start > 120` | The duration is over 120 minutes. Exactly 120 passes. | AC2 |
| 7 | `start < e and s < end` inside the loop | The candidate overlaps any existing `(s, e)`. Strict `<` means `e == start` or `s == end` is touching, not overlap. | AC4 |
| 8 | Final `return True` | Every check above passed. | AC5 |

**Why this order:**

- The type checks come first so that no comparison can raise a `TypeError`.
- The blocked check comes before the range and time checks because it overrides everything else.
- The `existing` scan comes last because it is the only step that costs more than constant time.

**AC5 non-mutation:** the function only iterates over `existing`. It never sorts, appends, removes or reassigns it. Three tests snapshot the list with `deepcopy` and confirm it is unchanged after `True`, `False`, blocked and invalid calls, including an unsorted list.

**Assumptions to keep in mind:**

- `existing` must be an iterable of `(s, e)` pairs. Passing `None` raises `TypeError`, as stated in the plan.
- `now` is trusted to be in 0..1439 and is only type-checked.
- Several tests set `now` and `existing` explicitly to isolate one rule. The base data (`now=540`, `blocked=False`, `[(600, 660)]`) applies by default.

Now proceed to Stage C (Critique) and Stage D (Repair) for `booking.py`:
1. Critically review our v1 implementation of `can_book` against AC1-AC5.
Specifically check:
- Does v1 properly reject boolean inputs for `start`, `end`, or `now` (since `isinstance(True, int)` is True in Python) or non-boolean values for `blocked`?
- Does v1 handle malformed tuples or non-integer values inside `existing` safely without mutating the input?
2. Propose a small, targeted preventive cleanup (v2) of `booking.py` and add corresponding unit tests in `test_booking.py` without expanding the scope.
3. Explicitly list what changed between v1 and v2.

Designing fail-closed validation for malformed existing intervals. >

Before judging v1, I'll run it against the inputs you asked about so the critique rests on observed behavior.

Edited 2 files, ran 2 commands, shared files >

All 50 tests pass on v2 (40 from v1 plus 10 new). The nonzero return code in that run came from `diff`, which exits 1 whenever files differ, not from the tests.

### Stage C: Critique of v1

I ran v1 against the inputs you listed instead of reasoning from the code alone.

**1. Booleans and non-bool `blocked`: v1 handles these correctly.**

- `can_book(True, 5, 0, ...)`, `end=True` and `now=False` all return `False`, because `_is_int` uses `type(value) is int` instead of `isinstance`.
- `blocked=1` returns `False` because of the `type(blocked) is not bool` check.
- No repair was needed here.

**2. Malformed `existing`: v1 has a real gap, in two forms.**

| Input in `existing` | v1 behavior | Problem |
|---|---|---|
| `None`, `5` (non-iterable) | raises `TypeError` | Violates AC5, which says `False` otherwise |
| `(600,)`, `(600,` | raises `ValueError` | Same |

Your free messages return at 1:20 AM. See Pro plan
Activate Windows
Go to Settings to activate Windows.

bad `start`, `end`, `now` and `blocked`, but not for bad `existing`.

**Non-mutation is not affected.** v1 only iterates `existing`, so no input, malformed or not, causes a mutation.

**One test-quality issue in v1.** In `test_existing_not_mutated_on_true`, I had a stray call with start 760 and end 760. It returned `False` because the interval was zero-length, not for the reason the test implied. I removed it.

### Stage D: v2 repair

**What changed in `booking.py`:**

1. **New helper `_is_valid_interval(item)`.** It accepts only a tuple or list of exactly 2 elements, both real ints. The same `type(x) is int` rule is used, so bools and floats are rejected.
2. **`iter(existing)` wrapped in `try/except TypeError`.** A non-iterable `existing` now returns `False`.
3. **The `for s, e in existing:` loop became `for item in intervals:`.** Each item is validated before it is unpacked, and a malformed item returns `False`. The overlap test `start < e and s < end` is unchanged.
4. **Docstring updated** to state the fail-closed contract.

**What did not change:** the order and logic of every other check (AC1 to AC3, duration, blocked), the return values for all well-formed input, and the single-pass read-only scan.

**Why return `False` instead of raising?** If `existing` cannot be read reliably, availability cannot be confirmed, so refusing the booking is the safe outcome. It also matches AC5 and the way v1 already treats the other inputs.

**What changed in `test_booking.py`:**

- **Fixed:** the misleading stray call in the non-mutation test, as noted above.
- **Added `TestMalformedExisting`, 10 tests:**
  - Non-iterable `existing`.
  - Wrong-length entries.
  - Non-sequence entries and strings.
  - Non-int values inside entries.
  - Bool values inside entries (the `(True, 5)` case).
  - A malformed entry placed after a valid one.

Your free messages return at 1:20 AM. See Pro plan
Activate Windows
Go to Settings to activate Windows.